

PHASE 3 — DOCKER & CONTAINERIZATION

DAY 12 — DOCKER COMPOSE

Tool: Docker Compose

1. Objectives

The objectives of Day 12 are:

- Understand Docker Compose services
 - Understand Docker Compose networks
 - Understand Docker Compose volumes
 - Understand environment variables
 - Understand healthchecks
-

2. Docker Compose Fundamentals

Docker Compose is used to define and run multiple containers as a single application stack.

A Compose configuration describes the services required by an application and how those services communicate with each other.

For this practical exercise, the application stack contains three services:

Nginx Reverse Proxy

|

v

Application

|

v

Database

3. Services

A **service** represents a container or application component managed by Docker Compose.

The Day 12 application contains three services:

App

The application service runs the Python web application on port 8000.

Database

The database service uses PostgreSQL to provide persistent application data storage.

Nginx

Nginx acts as the reverse proxy and receives requests on port 8080 before forwarding them to the application service.

4. Networks

Docker Compose networks allow services to communicate with each other.

For this project, two isolated networks are used:

frontend

```
|
+---- nginx
|
+---- app
```

backend

```
|
+---- app
|
+---- database
```

The application is connected to both networks.

Nginx is connected only to the frontend network.

The database is connected only to the backend network.

This provides network isolation between the reverse proxy and database.

5. Volumes

A Docker volume provides persistent storage outside the container's writable filesystem.

The PostgreSQL database uses the db-data volume:

volumes:

db-data:

The database mounts this volume at:

`/var/lib/postgresql/data`

This allows database data to persist independently of the database container.

6. Environment Variables

Environment variables are used to provide configuration values to containers.

The application uses:

`DB_HOST`

`POSTGRES_DB`

`POSTGRES_USER`

`POSTGRES_PASSWORD`

The database uses:

`POSTGRES_DB`

`POSTGRES_USER`

`POSTGRES_PASSWORD`

This allows the application to connect to the PostgreSQL database using the Compose service name db.

7. Healthchecks

A healthcheck allows Docker to determine whether a container's service is functioning correctly.

Healthchecks were configured for:

- Application
- PostgreSQL database
- Nginx reverse proxy

The application healthcheck checks the application on port 8000.

The database healthcheck uses `pg_isready`.

The Nginx healthcheck checks the Nginx HTTP service.

A healthy service is reported by Docker Compose as:

healthy

8. Hands-on Practice

8.1 Create the Project

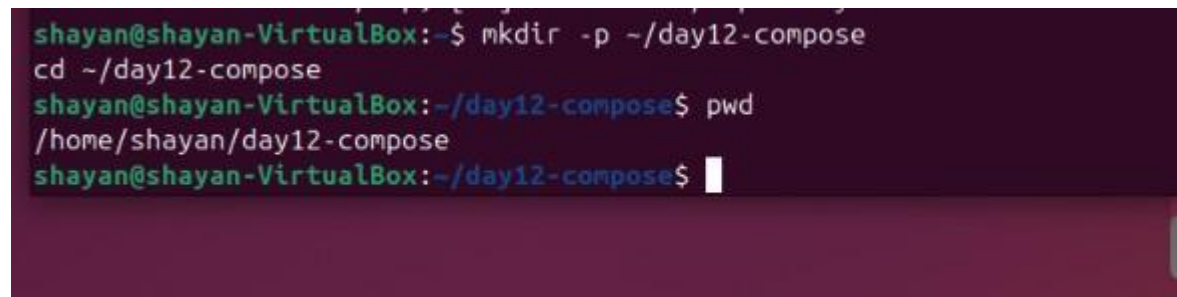
A dedicated project directory was created:

```
mkdir -p ~/day12-compose
```

```
cd ~/day12-compose
```

The project contains the application and Docker Compose configuration files.

Screenshot 1 — Project Files



```
shayan@shayan-VirtualBox:~$ mkdir -p ~/day12-compose
shayan@shayan-VirtualBox:~$ cd ~/day12-compose
shayan@shayan-VirtualBox:~/day12-compose$ pwd
/home/shayan/day12-compose
shayan@shayan-VirtualBox:~/day12-compose$
```

8.2 Create the Application

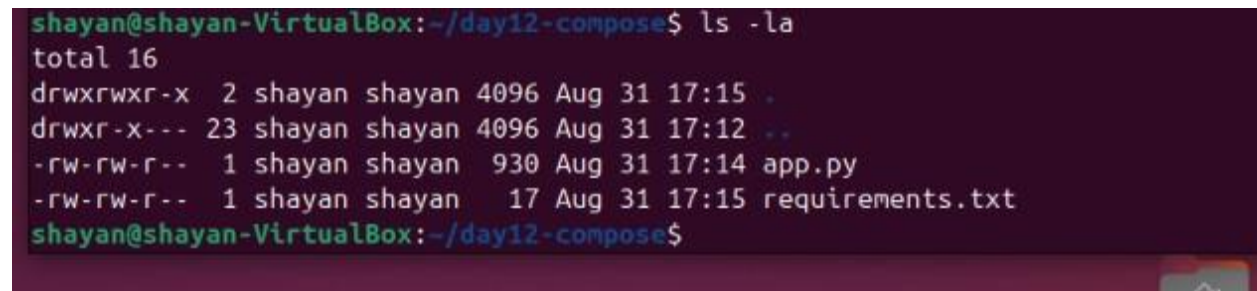
A small Python web application was created in app.py.

The application reads the database configuration from environment variables and attempts to connect to the PostgreSQL database.

The required Python dependency was defined in:

requirements.txt

Screenshot 2 — Application Files

A terminal window screenshot showing the command 'ls -la' being executed in the directory ~/day12-compose. The output lists the files: a directory '.', a directory '..', a file 'app.py' (930 bytes), and a file 'requirements.txt' (17 bytes). The user is shayan@shayan-VirtualBox.

```
shayan@shayan-VirtualBox:~/day12-compose$ ls -la
total 16
drwxrwxr-x  2 shayan shayan 4096 Aug 31 17:15 .
drwxr-x--- 23 shayan shayan 4096 Aug 31 17:12 ..
-rw-rw-r--  1 shayan shayan  930 Aug 31 17:14 app.py
-rw-rw-r--  1 shayan shayan   17 Aug 31 17:15 requirements.txt
shayan@shayan-VirtualBox:~/day12-compose$
```

8.3 Build the Application + Database + Reverse Proxy Stack

Docker Compose was used to define three services:

app

db

nginx

The Compose configuration connects the application to PostgreSQL and configures Nginx as the reverse proxy.

The stack was started using:

```
sudo docker compose -f compose.yaml up -d
```

The services were verified using:

```
sudo docker compose -f compose.yaml ps
```

Result

The application, database, and reverse proxy services were successfully created and started locally.

Screenshot 3 — Compose Services

```
shayan@shayan-VirtualBox:~/day12-compose$ nano compose.yaml
shayan@shayan-VirtualBox:~/day12-compose$ nano nginx.conf
shayan@shayan-VirtualBox:~/day12-compose$ ls -la
total 24
drwxrwxr-x  2 shayan shayan 4096 Aug 31 17:36 .
drwxr-x--- 23 shayan shayan 4096 Aug 31 17:35 ..
-rw-rw-r--  1 shayan shayan  930 Aug 31 17:14 app.py
-rw-rw-r--  1 shayan shayan  686 Aug 31 17:36 compose.yaml
-rw-rw-r--  1 shayan shayan  131 Aug 31 17:36 nginx.conf
-rw-rw-r--  1 shayan shayan   17 Aug 31 17:15 requirements.txt
shayan@shayan-VirtualBox:~/day12-compose$
```

8.4 Environment Variables

Environment variables were configured in `compose.yaml` for the application and PostgreSQL database.

The application uses:

`DB_HOST=db`

`POSTGRES_DB=day12db`

`POSTGRES_USER=day12user`

`POSTGRES_PASSWORD=day12pass`

These values allow the application to locate and authenticate with the PostgreSQL database service.

Screenshot 4 — Environment Variables

```
shayan@shayan-VirtualBox:~/day12-compose$ sudo docker compose -f compose.yaml ps
NAME                                IMAGE                                COMMAND                                SERVICE  CR
day12-compose-app-1                python:3.12-slim                   "sh -c 'pip install ...'"           app      14
seconds ago Up 11 seconds
day12-compose-db-1                 postgres:16                         "docker-entrypoint.s..."           db        16
seconds ago Up 13 seconds 5432/tcp
day12-compose-nginx-1              nginx:alpine                       "/docker-entrypoint...."           nginx    14
seconds ago Up 9 seconds 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
shayan@shayan-VirtualBox:~/day12-compose$
```

8.5 Volume Configuration

A named Docker volume called db-data was configured for PostgreSQL.

volumes:

db-data:

The volume is mounted to the PostgreSQL data directory:

/var/lib/postgresql/data

This provides persistent storage for the database.

8.6 Healthchecks

Healthchecks were added to all three services.

The application healthcheck verifies the application HTTP service.

The PostgreSQL healthcheck uses:

pg_isready

The Nginx healthcheck verifies the Nginx HTTP service.

The stack was restarted after adding the healthchecks:

```
sudo docker compose -f compose.yaml up -d
```

The health status was verified using:

```
sudo docker compose -f compose.yaml ps
```

Result

The services reported their health status through Docker Compose.

Screenshot 6 — Healthchecks



```
shayan@shayan-VirtualBox:~/day12-compose$ sudo docker compose -f compose.yaml ps
NAME                                STATUS      IMAGE           COMMAND                                SERVICE  CREATE
day12-compose-db-1                 Up 29 seconds (healthy)    postgres:16     "docker-entrypoint.s..."             db       33 sec
day12-compose-nginx-1              Up 26 seconds (health: starting)  nginx:alpine    "/docker-entrypoint..."             nginx    31 sec
```

8.7 Isolated Networks

Two Docker Compose networks were configured:

frontend

backend

Network assignment:

Nginx → frontend

App → frontend + backend

Database → backend

This isolates the database from direct communication with the reverse proxy.

The configured networks were verified using:

`sudo docker network ls`

Screenshot 7 — Isolated Networks



```
shayan@shayan-VirtualBox: ~/day12-compose$ sudo docker compose -f compose.yaml ps
NAME                                IMAGE                                COMMAND                                SERVICE  CR
EATED                                STATUS                                PORTS                                SERVICE  CR
day12-compose-app-1                python:3.12-slim                    "sh -c 'pip install ...'"           app      16
seconds ago Up 12 seconds (health: starting)
day12-compose-db-1                 postgres:16                          "docker-entrypoint.s..."           db        17
seconds ago Up 14 seconds (healthy)    5432/tcp
day12-compose-nginx-1              nginx:alpine                        "/docker-entrypoint...."           nginx     15
seconds ago Up 11 seconds (health: starting) 0.0.0.0:8080->80/tcp, [::]:808
0->80/tcp
shayan@shayan-VirtualBox: ~/day12-compose$
```